IMPLEMENTATION BLUEPRINT · DRAWING SET

Goal-Compiled Agentic MoE

A goal compiler, a goal-conditioned MoE hub and a compiler-grade trust layer, specified in eleven sheets.

A goal compiler turns each request into a typed task graph and a static schedule. Every step's executor, token budget, escalation ladder, guards and worst-case cost are decided before anything runs. An assembly of executors runs that schedule: compiled code, System-1 models, classic ML, a Qwen3.6-35B-A3B hub with goal-family capacity, and a Qwen3.8-27B spoke. Everything runs under a determinism contract with replay, receipts and declared guarantees per executor.

PROJECT	GCAM · Goal-Compiled Agentic MoE
PLAN	v6.2 · docs/PLAN.md
DATE	2026-10-21
HUB	Qwen3.6-35B-A3B (35B total, ~3B active)
REFERENCE	Qwen3.8-27B (dense)
REVIEW	Round 6: approved with changes, no blockers (all applied). Round 7 in progress.
REPOSITORY	gauravdc27/model-compiler
BRANCH	claude/magical-allen-eickyt
FORMAT	A4 landscape · 23 pages · vector drawings

DRAWING INDEX

01	System overview	4
02	Compiling a goal into a static schedule	6
03	Escalation ladders priced on joint calibration	7
04	Guards, variants and recompiles	8
05	Inside the hub: one MoE layer with goal capacity	9
06	Served artifact: A versus B	11
07	Training program and stage edges	12
08	Cost feasibility model	14
09	Trust layer: replay and contracts	16
10	Roadmap and gates	17
11	What exists in the repo	19

REVISION HISTORY

REV	CHANGE
v1–v4	Determinism contract D0–D⊥, requirements R1–R28, trust architecture; converged over review rounds 1–3
v5	Rethink as a goal-compiled agentic MoE on Qwen3.6-35B-A3B; Groq-style compile-time scheduling
v6	Feasibility-derived targets, served artifact A/B, G0 gates, agentic spike, allocation tiers (round 4)
v6.1	Per-suite cost targets, every-variant bound, roadmap follows stage edges, goal-suite size (round 5)
v6.2	Spike gate power, δ_b cap and pooling, switch rule on the spike checkpoint, gated confirmations (round 6)

QUALITY TARGET	COST TARGET	ESCALATION BUDGET	BUILT SO FAR
Non-inferior to Qwen3.8-27B	**≤ 0.5× reasoning · ≤ 0.6× agentic**	**≤ 15% of the 27B's tokens**	**Compiler core · 83 tests**

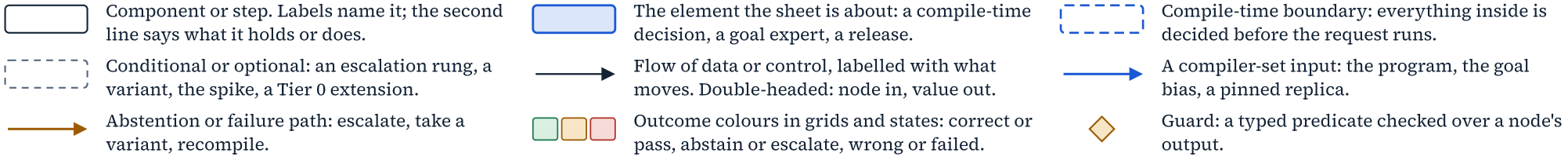
CONTENTS

How this set is organised

Source of truth: docs/PLAN.md v6.2

—	Executive summary Goal, approach, targets, decisions and open questions.	3
01	System overview Compiler, runtime, executor tiers and trust layer, and the arrows between them.	4
02	Compiling a goal into a static schedule An example goal laid out on two lanes with worst-case bars, a guard and a variant.	6
03	Escalation ladders priced on joint calibration Why ladders are priced item by item: the scheduler's own correlated-errors test.	7
04	Guards, variants and recompiles What happens when a guard fails, and how the program bound adds up.	8
05	Inside the hub: one MoE layer with goal capacity Router, goal bias, base, hot and goal experts in one MoE layer of the hub.	9
06	Served artifact: A versus B Per-family checkpoints versus one goal-gated checkpoint, and the rule that picks.	11
07	Training program and stage edges T0–T6, the agentic spike, and the edges that keep atlases and heads fresh.	12
08	Cost feasibility model Cost-of-pass as a function of the escalation budget, with the targets drawn in.	14
09	Trust layer: replay and contracts Replay versus recompute, the determinism levels, and each executor's contract.	16
10	Roadmap and gates G0–G4 with their gates, the two allocation tiers, and the G0 measurements.	17
11	What exists in the repo Modules, tests and the status of every CPU pilot.	19
A	Risk register Every Part I risk with likelihood and mitigation (PLAN.md §I.7).	21
B	Glossary Terms, symbols and stage names used on the sheets.	22
C	Review trail and evidence base Six adversarial review rounds and the sources behind every number.	23

Reading the drawings



Every drawing is one claim, stated above it and in its caption. Section references (§I.2.1, R26) point into PLAN.md; file names point into the repository. Numbers marked illustrative are worked examples; every other number is from the plan, the code's tests, the research notes or the A3B training report.

SUMMARY

Executive summary

Plan §I.0, §I.1

What is being built

An agentic model system that matches Qwen3.8-27B on reasoning and agentic work at a fraction of its cost, and that people can trust the way they trust a compiler. The base is Qwen3.6-35B-A3B, a 256-expert MoE with about 3B active parameters. The gap to close is mainly agentic: on its model card the hub trails the 27B by 3 points on GPQA-Diamond but by 12 on SWE-bench Pro.

How

- **Compile, then run.** A goal compiler fixes each step's executor, budget, escalation ladder, guards and worst-case cost before execution. This is Groq's discipline applied to agents: no runtime arbitration, one plan as the source of truth.
- **Cheapest adequate executor.** Code, System-1 heads, encoders and classic ML take whole steps off the LLM and abstain upward only when calibration says they are unsure. Ladders are priced on joint calibration data, because escalated items are the hard ones.
- **Goal-based MoE.** The goal selects capacity at compile time: hot-expert adapters per goal family (item i) and goal-gated new experts reached through a compiler-set router bias (item ii). With the goal off, a layer is bit-identical to the base.
- **Capability transfer.** On-policy distillation from Qwen3.8-27B over the shared 248,320-token vocabulary, agentic RL with routing replay, and decision-token DPO, on top of the user's proven A3B recipe.
- **Trust.** Replay is exact by construction. Each executor declares the determinism contract it delivers and each node the one it requires. Every call leaves a receipt.

Built so far

The scheduler (joint calibration, caps, guards, variants, recompiles, capability ceilings, confirmation for irreversible actions), goal-gated experts, hot-expert LoRA and the trust layer exist as tested Python (112 tests). GCAM v1 runs nine goal families end to end on CPU: compiled ladders match or beat a dense specialist on every decision op at under 0.2% of its cost on classification, and the hub's goal-family layers lift it from 0.27 to 0.96 and 0.50 to 0.98 on their families. Four CPU pilots are done and three are queued.

Targets (fixed before measurement)

Reasoning	Lower 95% CI of (GCAM − 27B) ≥ −δ_b on GPQA-D, HLE, AIME 24–26, LCB v6, MMLU-Pro (δ_b capped at 4; AIME and GPQA-D expected resolution-limited, so pooled into a composite)
Agentic	Same rule, floor 2 points: SWE-bench Pro, Terminal-Bench 2.x, τ^3-bench, BrowseComp, goal suite
Cost	≤ 0.5× cost-of-pass (reasoning, goal suite), ≤ 0.6× (agentic), escalation e ≤ 0.15
Latency	Mean ≤ 0.6×; p50 and p90 reported
Predictability	Worst-case bound by construction; realized within ±15% over ≥ 200 tasks

Decided, and open

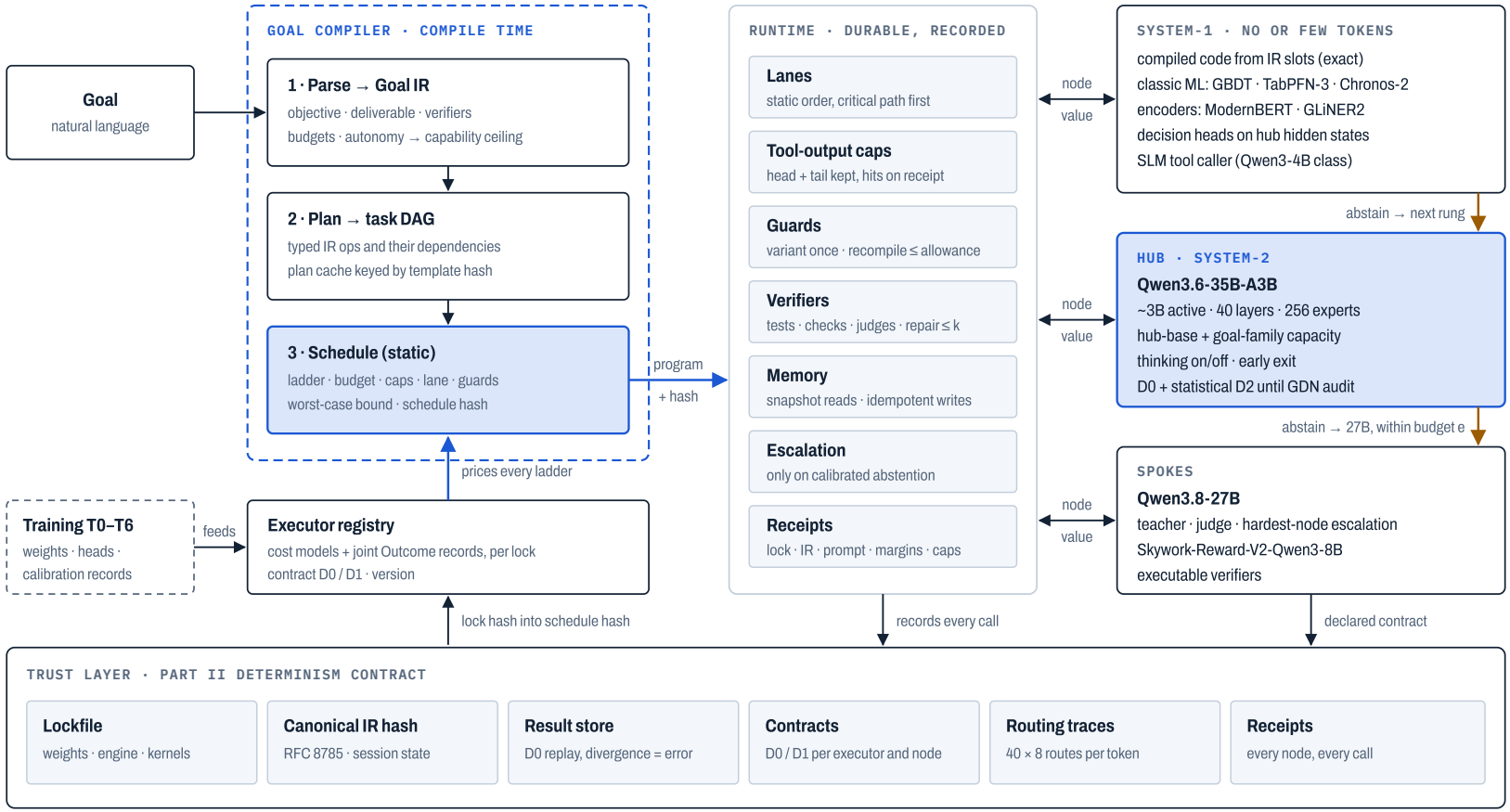
decided	Hub base, compile-time scheduling, joint calibration, per-family checkpoints served from G1, cost formula
open · G1	Does RL move agentic quality at 3B active? Agentic spike; switch to Qwen3.8-Flash-Next if not
open · G0	Hub cost per token r; prefix caching and determinism on Gated DeltaNet; thinking control; MTP
open · G2	One goal-gated checkpoint (B) instead of per-family checkpoints (A)
open · T2	Do new goal experts beat matched LoRA? Pilot 06 on CPU first

SHEET 01

System overview

Plan §I.0, §I.2

Decisions are made once, at compile time, from calibrated numbers. At run time the system only executes the schedule, checks guards, escalates on calibrated abstention and records everything.



The compiler (dashed blue) is the only place where executors, budgets and escalation paths are chosen. The runtime never re-plans on its own: it follows the schedule, takes pre-compiled variants on guard failure, and abstains when its allowance runs out.

Groq's lesson, transferred

Groq removes runtime arbitration: the compiler decides where and when every operation runs, so latency and cost are known in advance. Agents have data-dependent output lengths, so cycle-exact timing does not transfer. The discipline does: whole-plan analysis, reserved resources, one plan as the source of truth, and explicit guards where data-dependence cannot be avoided.

The user's five items

- (i) Train discovered experts → hot-expert adapters per goal family (Sheet 05).
- (ii) New layers and router → goal-gated experts with a compile-time goal bias (Sheet 05).
- (iii) Specialists and ensembles → spokes, cascades, verifier-selected best-of-K.
- (iv) System-One models → decision heads, encoders, SLMs and code that abstain upward.
- (v) Classic ML/DL → GBDT, TabPFN, Chronos-2, kNN and rerankers as registered executors.

Component specification

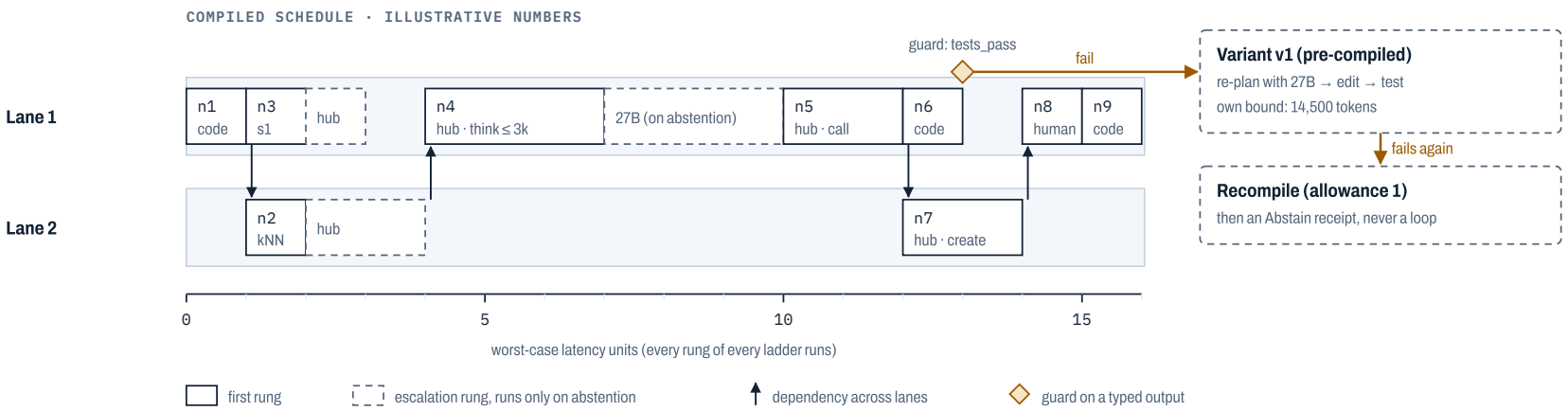
COMPONENT	RESPONSIBILITY	WHERE IT LIVES	CONTRACT
Parse	Natural language → Goal IR; ambiguity becomes Clarify, never a guess	Dense parser spoke (P2); rule parser v0 in `dmc/compiler.py`	D1 target
Planner	Goal IR → DAG of typed ops; plans cached by template hash	Hub Plan profile (G1)	D0
Scheduler	Ladders, budgets, caps, lanes, guards, variants, bound, schedule hash	`dmc/scheduler.py`	Pure function of IR, lock and cost models
Executor registry	Cost models and per-item calibration records per executor and lock	T6 output; `Executor`, `Outcome`	Versioned per lock
Runtime	Runs the schedule, enforces caps, evaluates guards, records activities and receipts	Durable executor (P3, Temporal-class); P1 is the deterministic serving lane	Records every call
System-1 tier	Code, classic ML, encoders, decision heads, SLM caller	T6	D1 on certified stacks
Hub	Reasoning, planning, tool calls, creation, with goal-family capacity	Qwen3.6-35B-A3B after T1–T5	D0 + statistical D2 until the GDN audit
Spokes	Teacher, judge and hardest-node escalation; reward model; verifiers	Qwen3.8-27B; Skywork-Reward-V2-Qwen3-8B	D0
Trust layer	Lock, canonical hashing, result store, receipts, routing traces	`dmc/lockfile.py`, `canonical.py`, `store.py`, `routing.py`	Defines D0–D⊥

SHEET 02

Compiling a goal into a static schedule

Plan §I.2.1 · dmc/scheduler.py

Example goal: “fix the failing date-parsing test in this repository and open a pull request.” The compiler places nine nodes on two lanes, sizes every bar at its worst case, and attaches a guard and a pre-compiled variant before execution starts.



Bars are sized at the worst case, so the schedule's finish time and token bound hold even if every ladder escalates. Expected cost is lower and comes from joint calibration (Sheet 03).

NODE	OP	LADDER	INPUT CAP	WORST-CASE TOKENS	CAPABILITY
n1	read issue	code	–	0	read
n2	locate files	embedding kNN → hub	2,000	3,000	read
n3	classify the failure	System-1 head → hub Decide	2,000	4,600	read
n4	plan the fix	hub thinking (≤ 3k) → Qwen3.8-27B	4,000	16,000	read
n5	edit code	hub Call, grammar-constrained	4,000	5,500	write
n6	run tests	code · guard `tests_pass`	–	0	read
n7	write PR text	hub Create	2,000	2,800	read
n8	confirm the send	`meta.confirm`; abstains if refused	–	0	read
n9	open the PR	code, only behind n8	–	0	external_send
	Main graph			**31,900**	ceiling: read · write · external_send

SHEET 03

Escalation ladders priced on joint calibration

Plan §I.2.1, §I.2.2 · Outcome, _estimate_joint

A cascade passes its hardest items upward. Multiplying each rung's average accuracy ignores that, so the scheduler replays every candidate ladder item by item on a shared calibration split.



This is the scheduler's own unit test (`test_joint_calibration_accounts_for_correlated_errors`). The independent estimate over-predicts accuracy by 5 points and under-predicts tokens by half; the joint estimate is exact on the calibration items.

Executor registration contract

- Input schema compiled from IR slots (feature schema for classic ML).
- Output is a typed IR value plus a conformally calibrated probability. No executor returns free text into the graph.
- Cost model: coverage, accuracy, forced accuracy, token bounds, input reads, latency, version, plus per-item `Outcome` records.
- Contract D0 or D1, a lock, and a minimum calibration size per (op, difficulty) bucket.

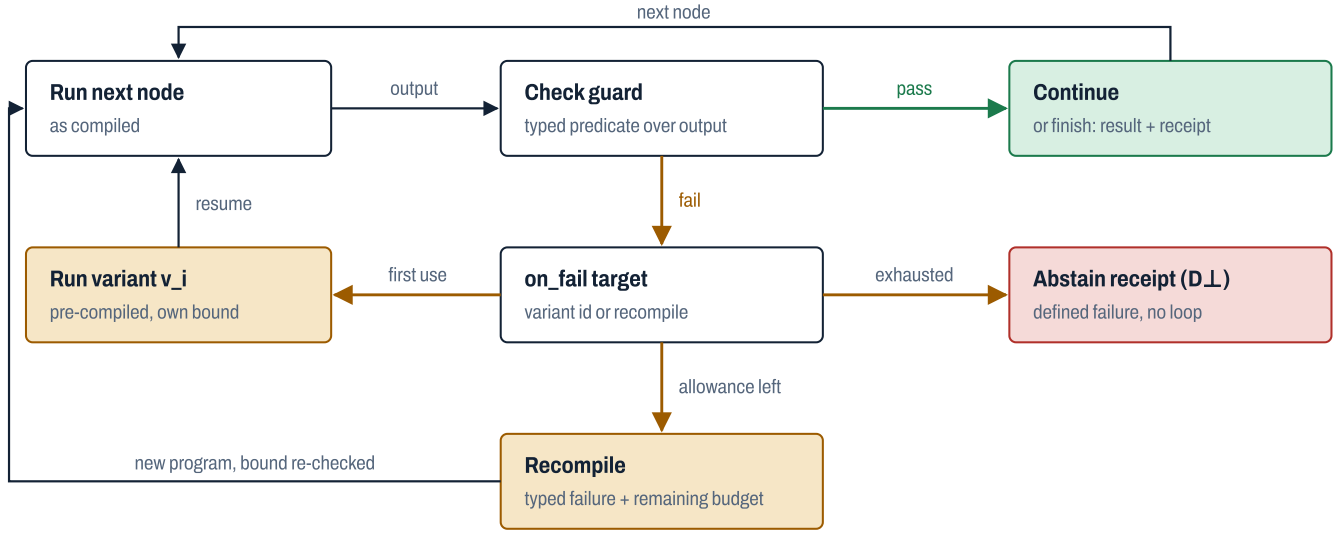
Bounds versus expectations

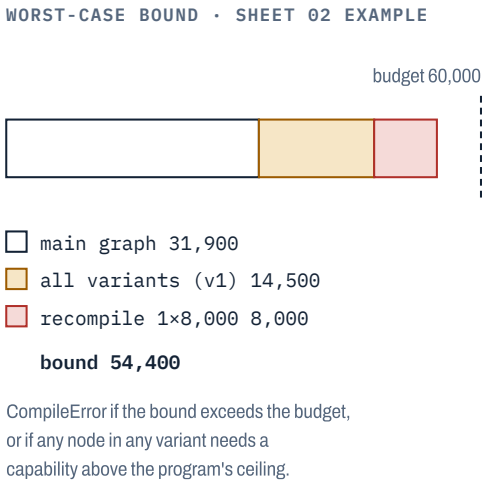
Executor token fields are compiled bounds: fixed prompt size, decode budget, and how many times the node's capped input is read (0 for code, 2 for two-order label scoring). Expected tokens come from what calls actually spent on calibration items.

Pilot 07 tests whether the joint prediction transfers to a held-out split. If it does not, bars are compared against the Wilson lower bound instead of the point estimate.

Guards, variants and recompiles

Data-dependent branches are handled the way a JIT handles deoptimization. Likely alternatives are compiled in advance, guards check typed outputs, and every path out of the plan is counted against the bound or ends in a defined failure.





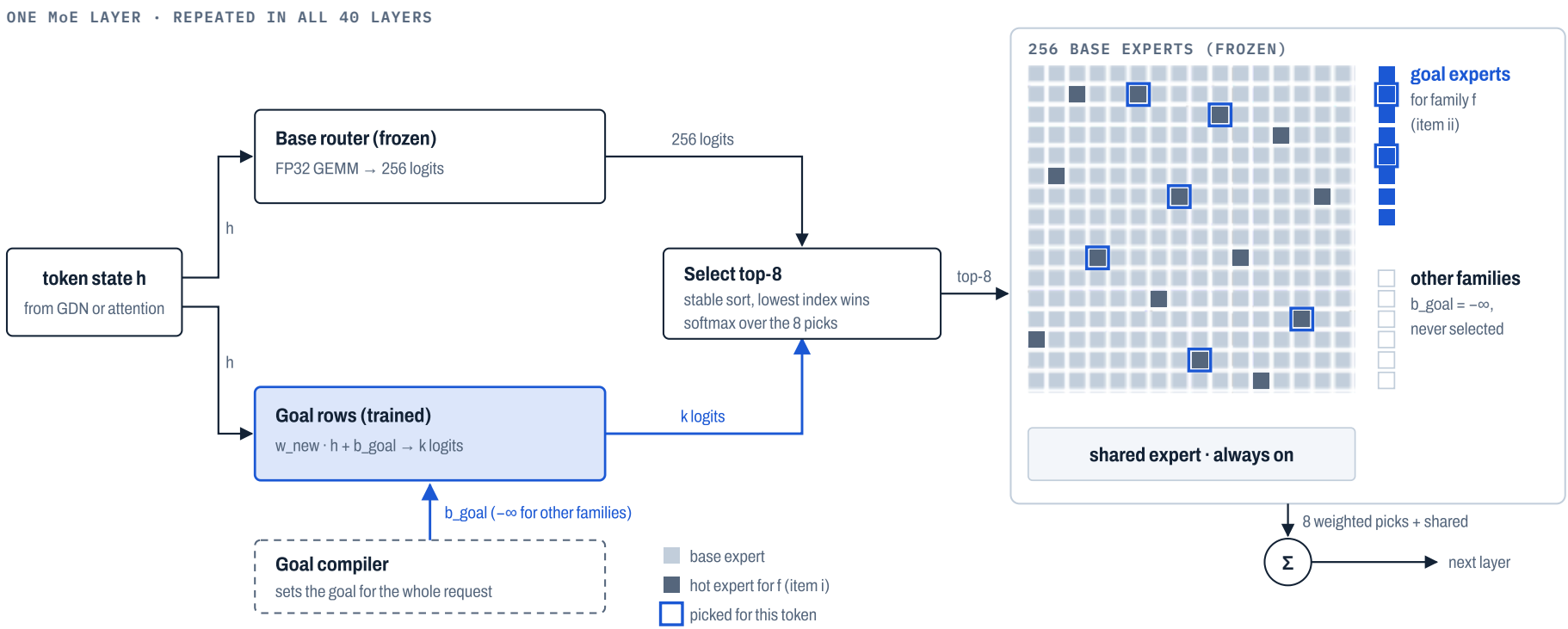
Each variant runs at most once and recompiles stop at the allowance. A guard inside one variant can hand off to another, so the bound counts every variant, not only the largest. Irreversible nodes (external_send, destructive) compile only behind a confirmation or dry-run node whose guard abstains when the confirmation is refused. Capabilities are fixed per branch at compile time, so a failed guard cannot steer execution into a branch with more authority than the goal was compiled with. This is the defence against branch-steering attacks on static plans.

SHEET 05

Inside the hub: one MoE layer with goal capacity

Plan §I.2.3 · dmc/expansion.py, dmc/expert_lora.py

The goal enters the hub as a bias on the router, set by the compiler before the request runs. For the goal's family it opens trained goal experts. For every other family the bias is −∞, so those goal experts are never selected. With no goal set, the layer is the base layer itself and its output is bit-identical by construction.



Routed variant shown: goal experts compete inside the top-8, so active compute does not change. In this token, two goal experts displaced two base experts. The additive variant instead adds gated goal experts on top of the 8 picks, at about +11% MoE-FFN compute per added expert.

Item (i): train the discovered experts

A routing atlas on each family's traffic, measured on hub-base, finds the hot experts by selection counts (A3B rule: 80% of overall mass ∪ 50% per slice; about 97 of 256 per layer for voice). They get a rank-16 LoRA whose B matrix starts at zero, so attaching it changes nothing until training moves it. Ablations: experts-only, and ESFT-tight sets of 5–15% of experts.

Item (ii): new experts and router

New experts are warm-started as copies of the family's hottest expert, with router rows copied from the parent. Noise ($\sigma = 0.05 \cdot \text{std}$) goes on the copy only, so the parent is never perturbed. Only the new experts, their rows and the goal biases train. A router-only review pass on less than 1% replay protects retention (MoE-LPR: 96.6% with it, about 87% without).

Guarantee, and how it is tested

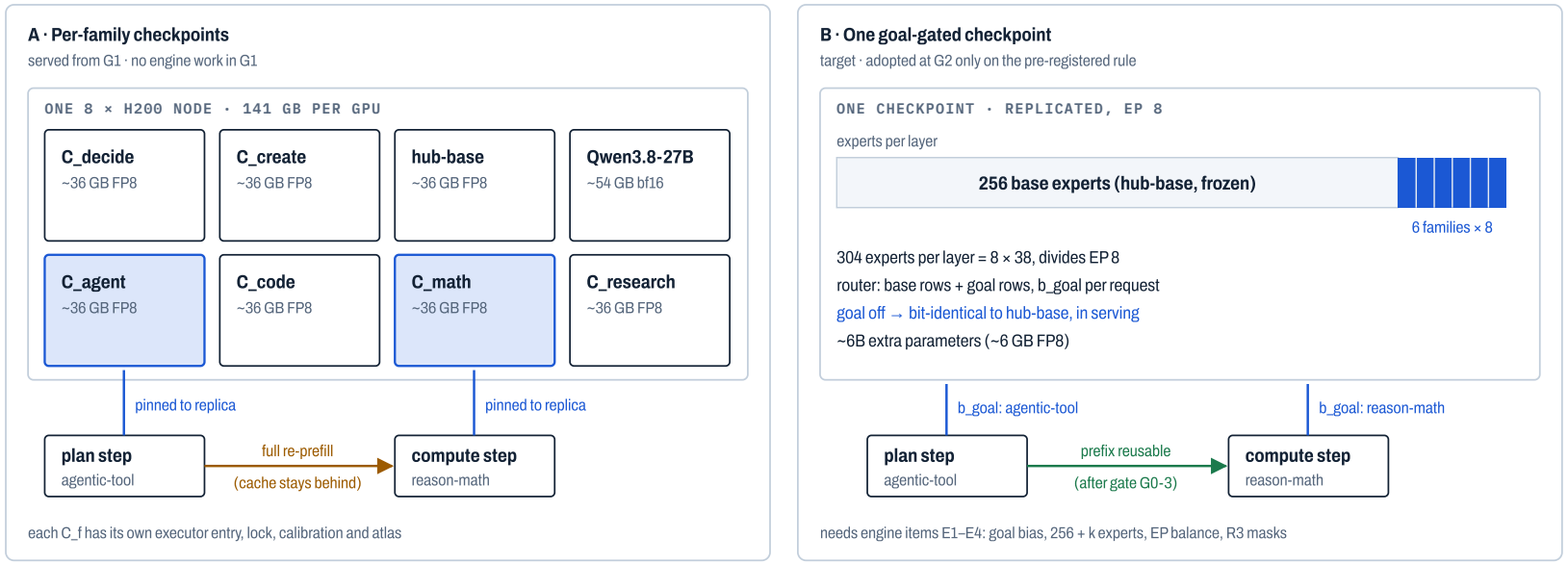
With the goal off, the wrapped block returns `base(h)` itself, so the result is bit-identical by construction (`tests/test_expansion.py`). Pilot 06 compares routed and additive experts against a matched-rank LoRA arm, a router-only arm and a random-parent arm, and also measures accuracy when the goal is wrong.

Hub specification

PROPERTY	VALUE	SOURCE
Parameters	35B total, ~3B active per token	Qwen3.6-35B-A3B model card (W2-05)
Layers	40 = 10 × (3 Gated DeltaNet + 1 gated attention), each followed by an MoE block; hidden size 2048	Model card, config (W2-03, W2-05)
Experts	256 routed (top-8) + 1 shared; expert FFN width 512	Config (W2-03)
One expert	3 × 2048 × 512 ≈ 3.15M parameters per layer; ≈ 126M across 40 layers (≈ 252 MB bf16)	Computed from the config (W2-03)
Rank-16 LoRA on one expert	≈ 0.09–0.12M parameters; ~97 hot-expert LoRAs per layer ≈ 3–4 full experts	W2-03
Goal experts at k = 48 (Artifact B)	≈ 6.0B parameters (≈ 6 GB FP8); 304 experts per layer = 8 × 38 for EP 8	Plan §I.2.3
Vocabulary	248,320 tokens, shared with Qwen3.8-27B (full-logit distillation is possible)	Model cards (W2-05)
Context; MTP	262,144 tokens native; multi-step MTP head (speedup on GDN unmeasured, G0-5)	Model card (W2-05)
Checkpoint size	~72 GB bf16, ~36 GB block-FP8 after merge	A3B report ch.2

Served artifact: A versus B

Item (i) merges adapters into weights, while item (ii) gates experts by goal. The plan serves per-family checkpoints first (A). It moves to one goal-gated checkpoint (B) only if B passes a comparison written down in advance.



The edge between the two task nodes is the difference. In A it crosses checkpoints and re-prefills. In B it stays inside one model and only the goal bias changes. Prefix reuse on the Gated-DeltaNet hybrid is disabled in production today, so neither option takes credit for it until gate G0-3 passes.

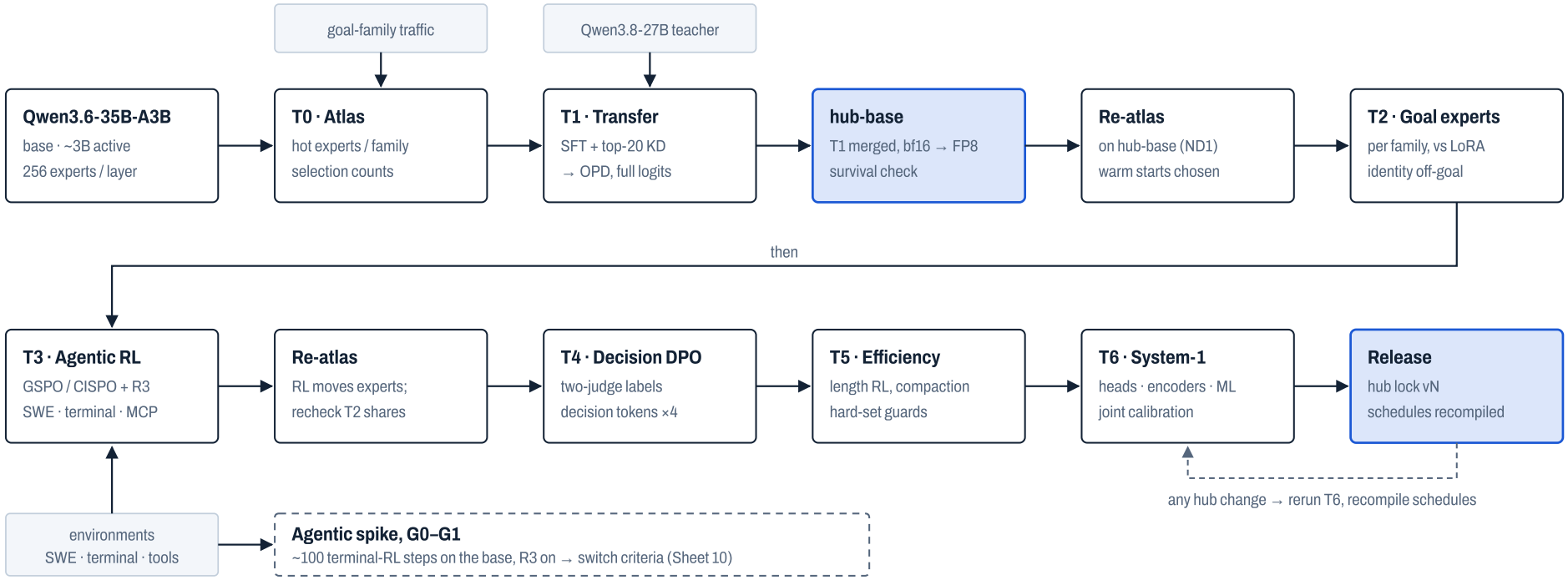
	A · PER-FAMILY CHECKPOINTS	B · ONE GOAL-GATED CHECKPOINT
Item (i)	Hot-expert LoRA merged into each family checkpoint	Expressed as item (ii): discovered experts trained as goal-gated copies, never in place
Goal off	Node is not routed to that checkpoint	Bit-identical to hub-base inside the serving engine
Engine work	None in G1; static bias and 256 + k experts (E2) once goal experts join in G2	E1 per-request goal bias · E2 256 + k experts · E3 EP load balance · E4 R3 routing-replay masks
Memory	~36 GB FP8 per family; up to six families per node	~6 GB FP8 extra at k = 48
Decision	Adopt B at the end of G2 if, on each of the first three families, B is non-inferior to A (δ_b on the family suite), off-family outputs equal hub-base bit for bit in serving, and E1–E4 pass the conformance matrix. Multi-LoRA serving (option C) is rejected.	

SHEET 07

Training program and stage edges

Plan §I.4

Training changes what the router sees, even with the router frozen. Every stage that reads expert usage or hub states therefore runs after the stage that last changed them.



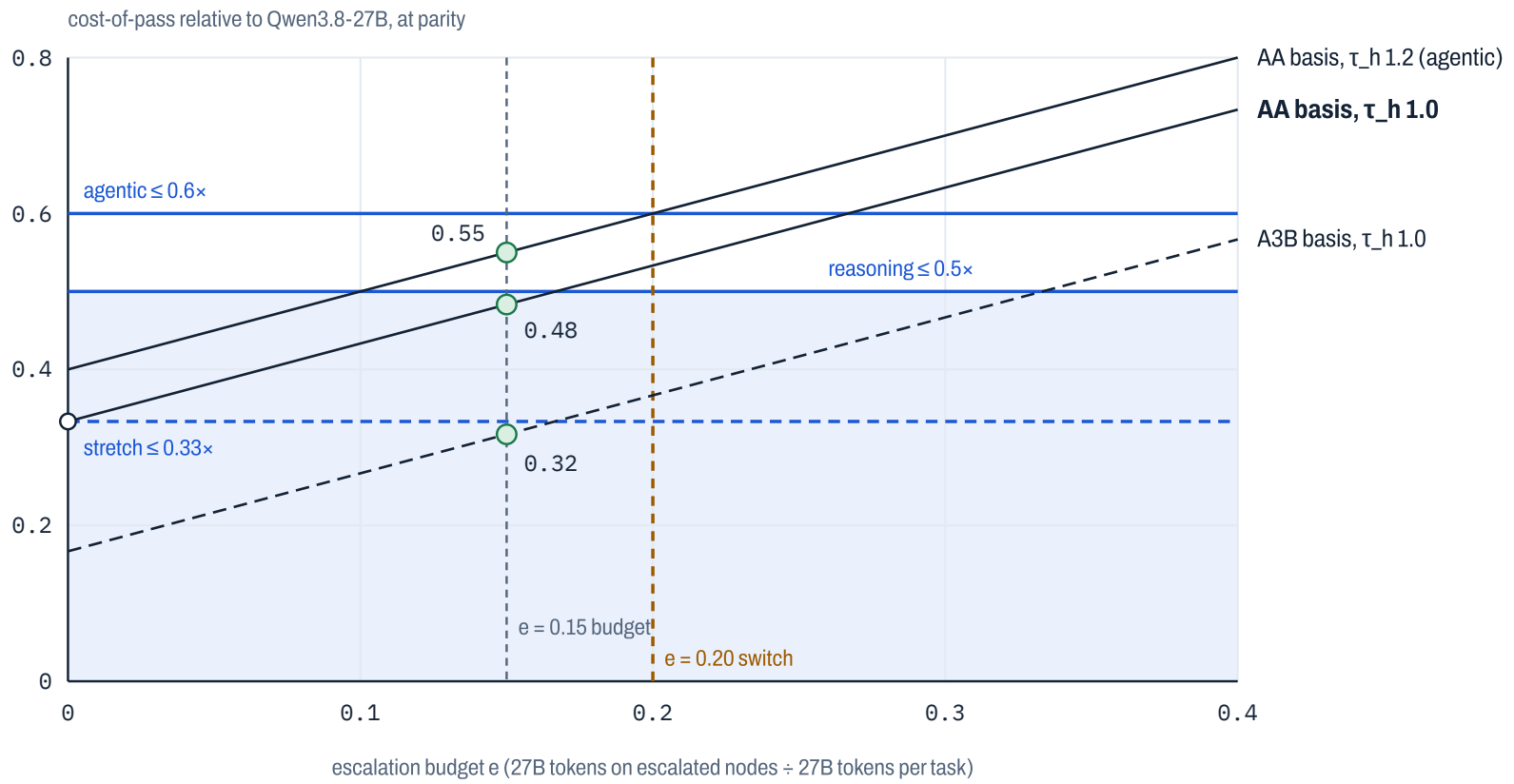
Arrows mean “must finish before”. T4 and T5 may run a first version on hub-base before T3 to test their recipes, but only the versions re-run after T3 ship. The spike (dashed) runs early on the untrained base because it tests the plan's riskiest assumption: that RL moves agentic quality at ~3B active parameters.

STAGE	RECIPE AND GUARDRAILS	COMPUTE
T0 Atlas	Routing atlas per goal family; selection counts, not router affinity (A3B: affinity entropy 0.98, selection concentrated); super-expert profile	< 50 GPU-h
T1 Transfer	SFT with top-20 KD (w = 0.3) on 27B traces, then on-policy distillation with full logits over the shared 248,320-token vocabulary; hot-expert LoRA, cached prefix + 512-token gradient tail, lr 1e-4; merge, FP8, survival check	1–3K H100-h

STAGE	RECIPE AND GUARDRAILS	COMPUTE
Spike	~100 terminal-RL steps with R3 on public terminal tasks (AgentWorld moved a 35B-A3B Terminal 32.8 → 47.0). Evaluated on ≥ 300 held-out terminal tasks with k = 8 samples each, so that SE ≤ 1.4 points. Pass: CI lower bound > 0 (≈ 80% power at a true +4). Switch signal: CI upper bound < +4. Otherwise inconclusive.	1 node × ~1 week
T2 Goal experts	Per family: gain against a matched-parameter LoRA arm, off-goal identity, traffic share, drift and wrong-goal accuracy	10–100 GPU-h / family
T3 Agentic RL	GSPO/CISPO sequence-level objectives, R3 routing replay, exact token handoff; SWE, terminal, tool simulators, browsing	Tier 1 ≤ ≈ 26K · Tier 0 ≤ 10K GPU-h
T4 Decision DPO	v1 on hub-base, re-run after T3. Two independent judges must agree; margin only on decision tokens (weight 4, β 0.5) plus anchors	< 100 GPU-h / round
T5 Efficiency	v1 on hub-base, re-run after T3. Difficulty-aware length RL, learned thinking on/off, CompactionRL, early-exit probes; hard-set non-inferiority guards	1–2K H100-h
T6 System-1	Heads, encoders, SLMs and GBDTs from hub/27B labels; conformal thresholds; joint `Outcome` records for every executor	CPU – small GPU

Cost feasibility model

At accuracy parity, cost-of-pass relative to the 27B = r·τ_h + e + s, where s, the cost of System-1 executors and code, is close to 0. The hub's per-token cost r is measured in G0. τ_h is hub verbosity relative to the 27B, and e is the escalation budget. The targets are read off this chart, not asserted.

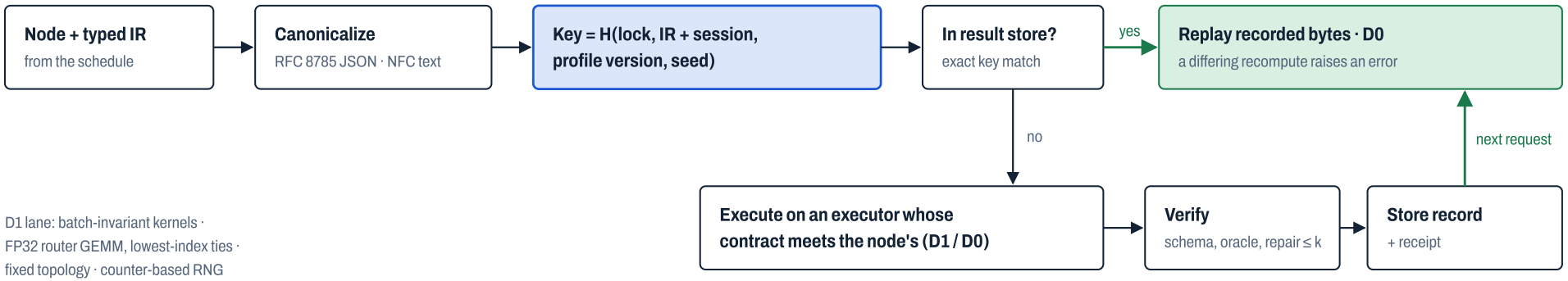


Shaded: meets the 0.5× reasoning target. Priors for r are 1/3 (Artificial Analysis throughput, 134 vs 44 tokens/s) and 1/6 (A3B GPU time per turn, 0.075 vs 0.449 s). Agentic suites plan for 20% more hub tokens, so their target is 0.6×. That line crosses 0.6 exactly at e = 0.20, which is where the base-switch criterion sits; between 0.15 and 0.20 the hub stays and the miss is reported. If G0 measures r > 0.4, the targets are re-derived from the same formula by a dated amendment before G1. Prefix-cache savings are not credited.

REQUIREMENT	VALUE	MEASURED AS
Reasoning parity	Lower 95% CI of (GCAM − 27B) ≥ −δ_b, with δ_b = max(1.5 points, one item, 2.8·SE_b), where SE_b is the paired SE of (hub-base − 27B), fixed at G1. δ_b is capped at 4 points; a benchmark above the cap (expected: AIME, GPQA-Diamond) is reported as resolution-limited and pooled into a reasoning composite	avg@16, paired item bootstrap; AIME 2024–2026 pooled (90 items)
Agentic parity	Same δ_b rule, floor 2 points; SWE-bench Pro, Terminal-Bench 2.x, τ^3-bench pass^4, BrowseComp, goal-suite v0	pass^k and pass@1; 27B-vs-itself null control
Cost	Compiled system with e ≤ 0.15: ≤ 0.5× cost-of-pass on reasoning suites and the goal suite, ≤ 0.6× on agentic suites; raw hub ≤ 1.0× tokens on reasoning at parity	Measured r; input, cached and output tokens; fleet-level cost under Artifact A
Latency	Compiled system mean ≤ 0.6×; p50 and p90 reported	Wall clock on the same node
Predictability	Bound holds by construction (main + every variant + recompile reserve, with capped tool outputs); realized mean within ±15% over ≥ 200 distinct tasks	Program bounds; joint-calibration audit with cap-hit rate

Trust layer: replay and contracts

The same canonical input, session state and lock always produce the same bytes. A first execution runs only on an executor whose contract meets what the node requires, and it is recorded so later requests replay it.



Replay is exact by construction. Bitwise recompute (D1) needs certified kernels. Until the hub's Gated-DeltaNet kernels pass a batch-invariance audit, nodes that require D1 are scheduled onto code, System-1 heads or dense students.

LEVEL	PROMISE
D0	Same canonical IR + session state + lock → byte-identical response (replay)
D1	Byte-identical recompute under any load, batch mix, chunking or cache state
D2	Decision fields agree across certified stacks at a stated coverage (statistical)
D3	Meaning-preserving paraphrases → same IR → same output (statistical)
D4	Output satisfies its schema and, where one exists, its executable oracle
D⊥	Bad or ambiguous input → CompileError, Clarify or Abstain, never a silent guess

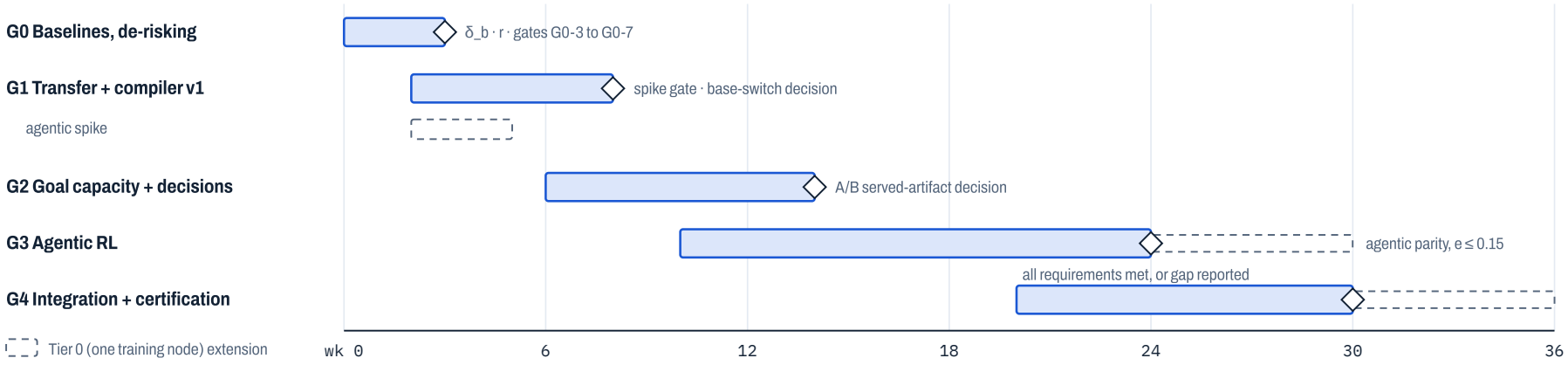
EXECUTOR	CONTRACT TODAY
Compiled code	D1 (exact)
System-1 heads, encoders, classic ML	D1 on their certified stack
Dense students	D1 candidates, certified per lock
Hub (Qwen3.6-35B-A3B)	D0 + statistical D2 until the GDN audit passes
Qwen3.8-27B spoke	D0 (record and replay) until its GDN audit passes
Hosted or closed models	D0 only, flagged on the receipt

SHEET 10

Roadmap and gates

Plan §I.6, §I.4 allocation tiers

The riskiest assumptions are tested first. Measurements that the later gates depend on (δ_b, the cost basis r, the prefix-cache gate, thinking control, MTP on GDN) all land in G0. The roadmap follows the stage edges of Sheet 07: hub-base is re-atlased inside G1 before any family checkpoint is built.



Tier 1 is two 8×H200 training nodes from week 2 to week 24, plus an evaluation node. Tier 0 is a single training node. With Tier 0, T3 runs as AgentWorld-scale RL runs capped at 10K GPU-hours, and G3 and G4 each end six weeks later. The system-level gates do not move.

G0 GATE	WHAT IS MEASURED	IF IT FAILS
G0-1	δ_b per benchmark: provisional from the paired SE of (base − 27B) at avg@16, fixed at G1 from hub-base; capped at 4 points	Above the cap: reported as resolution-limited and pooled into a reasoning composite
G0-2	Hub cost per token r on our node, input and output separately	Cost target re-derived by dated amendment
G0-3	Prefix caching on the Gated-DeltaNet hybrid; cached, uncached and repeat probe beyond 8.6k-token prefixes	No KV-reuse credit; prefix groups stay a placement preference
G0-4	Accuracy versus thinking budget under hard truncation; template and KV effect of per-turn thinking toggles	Compiler keeps thinking on/off + early exit only
G0-5	MTP acceptance and speedup on GDN (state rollback)	MTP stays off
G0-6	Batch-invariance audit of the hub's GDN kernels	Hub stays D0 + D2; D1 nodes use other executors

G0 GATE	WHAT IS MEASURED	IF IT FAILS
G0-7	Goal-suite v0: first 100 goals with executable verifiers or two-judge rubrics; ≥ 250 by week 6 (≥ 200 development, 20% held out)	G1 cannot report goal-suite numbers

Base-switch rule, fixed now. (a) The spike's gain has an upper 95% bound below +4 points. (b) On the spike checkpoint, the measured agentic cost $r \cdot \tau_h + e$ exceeds 0.6 (at the planning values, e above 0.20). Switch to Qwen3.8-Flash-Next if (a), or if the spike is inconclusive and (b) holds. If the spike passes and (b) holds, stay and re-check (b) on the T3 checkpoint at week 17. An escalation share between 0.15 and 0.20 fails the G3 gate and is reported, but does not trigger a switch.

SHEET 11

What exists in the repo

Plan Part III, Part IV

The compile-time machinery, the hub extensions and GCAM v1 exist as tested Python, with 112 tests in the suite. GCAM v1 runs nine goal families end to end on CPU, and every part of the architecture carries at least one of them. The CPU pilots run on granite-3.1-1b-a400m and are pre-registered before they run.

MODULE	WHAT IT DOES	SHEETS	STATUS
`dmc/scheduler.py`	Joint-calibration ladders, compiled caps and input reads, executor-keyed prefixes, worst-case bounds, lock-sensitive schedule hash, guards/variants/recompiles with an every-variant bound, capability ceiling, confirmation dependency for irreversible nodes, `cap_input` (runtime wiring pending)	02 03 04	implemented
`dmc/expansion.py`	Goal-gated experts, routed and additive, parent-row init, goal-off identity	05	implemented
`dmc/expert_lora.py`	Hot-expert LoRA on fused expert weights, identity at init	05	implemented
`dmc/routing.py`	FP32 router GEMM, lowest-index tie-break, token-verified route tracing and pinning, fragile-routing share	09	implemented
`dmc/ir.py` · `compiler.py` · `canonical.py` · `store.py` · `lockfile.py`	Typed IR (12 ops), pipeline and dispatch, canonical hashing, content-addressed replay and receipts, toolchain lock	01 09	implemented
`dmc/runtime.py` · `http_backend.py` · `profiles/decide.py`	Deterministic CPU backend, GPU backend over HTTP, order-averaged case-marginalized label scoring	09	implemented
`gcam/parser.py` · `planner.py` · `runtime.py`	GCAM v1: versioned goal grammar (9 families, Clarify on missing slots), plan templates with guards and variants, runtime with capped inputs, calibrated escalation, receipts and D0 replay	01 02 03	implemented
`gcam/executors/` · `gcam/system.py`	GCAM v1 executors: compiled code and verifiers, GBDT / Ridge / TF-IDF, MiniLM heads and a hub-state probe, the MoE hub with goal-gated experts (refund family) and hot-expert LoRA (support family), the dense spoke	04 05 06	implemented
`experiments/gpu/`	Engine-conformance script and runbook for GPU Phase 0	09 10	not run yet

PILOT	QUESTION	STATUS	RESULT SO FAR
01 Router atlas	How fragile is MoE routing?	done	7.8% of routing decisions have margin < 0.01; margin predicts flips (AUC 0.87)
02 CPU conformance	What survives a numerics change?	done	FP32 vs BF16: greedy text identical in 68% of prompts, scored decisions agree on 96.7%
05 Proxy calibration	Is the CPU thread-count proxy realistic?	done	Stress regime (mean KL 13.7× the published cross-engine value): CPU pilots can remove levers, not set production thresholds
03 Route pinning	Does replaying recorded routes beat fresh execution across stacks?	done	Not significant at n = 300: decision parity 96.0% pinned vs 93.3% fresh, 16 repaired vs 8 introduced (p = 0.076); R7 hardening removed almost nothing under the CPU proxy
06 Goal-gated experts	Do new experts beat matched LoRA, with identity off-goal?	queued	—

PILOT	QUESTION	STATUS	RESULT SO FAR
07 System-1 cascade	Does the joint schedule predict realized accuracy and tokens?	queued	—
04 Margin regularization	Can training widen router margins?	queued	—

GCAM V1 ON CPU (HELD-OUT ITEMS)	COMPILED LADDER	ACCURACY · COST	ALWAYS SPOKE
Refund-claim reason	keywords → encoder	0.963 · 1	0.95 · 777
Tool selection	keywords → encoder	0.950 · 1	0.825 · 959
Reply judging	executable verifiers	1.00 · 0	0.95 · 1,327
Expense review	GBDT → System-1 memo head	0.917 · 0.3	0.33 · 1,446
Cost estimate	arithmetic grammar → spoke program-of-thought	0.933 · 174	0.70 · 337
Policy retrieval	TF-IDF → encoder → spoke	1.00 · 1,112	0.92 · 2,956
Hub goal-gated experts (claims)	base hub 0.271 → 0.958 with the family on; goal off is bit-identical		
Hub hot-expert LoRA (tools)	base hub 0.50 → 0.979; with LLM rungs only, the compiler picks these layers over the spoke		
18 end-to-end goals	all within their compiled bounds; confirmation gates the destructive tool; D0 replay exact at zero cost		

Cost is in hub-token equivalents per item (spoke tokens weigh 3.75, encoder 0.057, code and classic ML 0). Details and what did not work: `docs/GCAM_V1.md`.

APPENDIX A

Risk register

Plan §I.7 (Part I) · Part II §II.8 covers the trust track

RISK	LIKELIHOOD	MITIGATION
Agentic gap does not close at 3B active (AA index 18 vs 34; same-generation agentic gap large)	Medium–High	Agentic spike in G0–G1, not G3; escalation within the budget e; switch criteria to Qwen3.8-Flash-Next fixed now (§I.2.3), with its memory (125B + 51B host-RAM n-gram tables), cost basis and license checked before a switch.
Cost target infeasible at the measured cost basis	Medium	The feasibility model is a formula; G0-2 plugs in r and the target is re-derived by dated amendment before G1, never after seeing GCAM results.
Prefix caching stays broken on the GDN hybrid	Medium–High	No KV-reuse credit anywhere until G0-3 passes; prefix groups remain a placement preference.
Distillation copies teacher mistakes; models agreeing ≠ correct (A3B lesson 4)	Medium	Two-judge labels for decision training; verifiers for reasoning; on-policy distillation only on verifiable or judged data.
Goal-gated experts starve or drift	Medium	Parent-row init with symmetric noise + review pass (W2-03); traffic-share, drift and wrong-goal metrics per family; off-goal identity guaranteed by construction on hub-base.
Artifact B does not match Artifact A	Medium	A is served from G1 and remains the default; B is adopted only on the pre-registered rule.
Engine work (E1–E4, R27) slips	Medium–High	Artifact A needs none in G1; budgeted with headcount; E3 (EP hot-spotting) carried with redundant-expert replicas.
LoRA-on-experts serving fragility	High	Merge adapters (Artifact A) or express them as goal-gated experts (Artifact B); multi-LoRA serving rejected.
Thinking control or MTP behaves differently on Qwen3.6 than on Qwen3	Medium	Measured in G0 (G0-4, G0-5) before the compiler relies on them; compiler v1 uses on/off + early exit only.
Atlases, warm starts and cost models go stale after training	High	Explicit stage edges (§I.4): re-atlas after T1 and T3, T6 last and per hub release, schedule hash includes the lock.
Static plans attacked via branch steering (COBRA)	Medium	Typed guards over data, capability ceilings fixed per branch at compile time, static escalation budgets.
Compaction drops safety rules	Medium	Pin rules outside compaction; safety evals after every compaction policy change.
Token-efficiency training lowers hard-set accuracy or CoT faithfulness	Medium	Non-inferiority guards on hard sets; faithfulness probes; per-node thinking decisions instead of global pressure.
Training allocation smaller than requested	Medium	Tier 0 plan with lowered raw-hub expectations and a later G3/G4; the system-level gates do not move.
Benchmark conflicts (AA index versions, harness differences)	High	Run every comparison ourselves, paired, same harness.
GDN determinism unknown	High	Contract-aware scheduling (D1 nodes avoid the hub until the audit passes); G0-6.

APPENDIX B

Glossary

Symbols and names used on the sheets

A3B report
The user's training report for Qwen3.6-35B-A3B voice agents (hot-expert LoRA, top-20 KD, decision-token DPO, two-judge labels). Its methods are carried over.

Artifact A / B
What is served. A: one merged checkpoint per goal family, pinned to its own GPU. B: one checkpoint whose goal-gated experts carry every family. A from G1; B only if it passes a comparison at G2.

avg@k
Accuracy averaged over k samples per item; reduces sampling noise in paired comparisons.

b_goal
Per-request router bias set by the compiler: opens a family's goal experts, −∞ for every other family.

Cost-of-pass
Cost per solved task. At accuracy parity its ratio to the 27B is r·τ_h + e + s (s: System-1 and code overhead).

D0 … D4, D⊥
Determinism levels: replay, bitwise recompute, cross-stack decision parity, paraphrase invariance, spec conformance, defined failure.

δ_b
Non-inferiority margin for benchmark b: max(1.5 points, one item, 2.8 × paired SE of hub-base − 27B), capped at 4 points; benchmarks above the cap are pooled.

e
Escalation budget: tokens the 27B spends on escalated steps ÷ the 27B's tokens per task. Planned ≤ 0.15.

ESFT
Expert-specialized fine-tuning: tuning only the 5–15% of experts a task uses.

G0 … G4
Roadmap phases: baselines and de-risking, transfer and compiler v1, goal capacity, agentic RL, certification.

GDN
Gated DeltaNet, the linear-attention layers in 30 of the hub's 40 layers. Their determinism and prefix caching are G0 gates.

Guard, variant, recompile
A guard is a typed check on a step's output. On failure the runtime runs a pre-compiled variant once, or recompiles within an allowance, or abstains.

Hot experts
The experts a goal family's traffic selects most, found by selection counts in a routing atlas.

hub-base
Qwen3.6-35B-A3B after T1 capability transfer, merged and exported to FP8; every family's capacity is trained on it.

Joint calibration
Running every executor on the same calibration items and pricing a ladder by replaying it item by item (Outcome records).

Lock, receipt
The lock pins every output-affecting artifact; its hash enters every result key. A receipt records how each result was produced.

MTP
Multi-token prediction head used for speculative decoding; unmeasured on GDN, off in the bitwise lane.

OPD
On-policy distillation: the student samples, the teacher scores every token.

r
The hub's serving cost per token over the 27B's, measured on our node in G0. Priors 1/3 and 1/6.

R3
Routing replay: rollout routes reused in the RL update so MoE training stays stable.

System-1 / System-2
Fast executors that answer without generating tokens, versus the hub and spokes that reason in tokens.

T0 … T6
Training stages: atlas, transfer, goal experts, agentic RL, decision DPO, efficiency, System-1 distillation.

τ_h
Hub tokens per task ÷ the 27B's. Planned ≤ 1.05 on reasoning, 1.2 on agentic suites.

APPENDIX C

Review trail and evidence base

docs/reviews/ · research_notes/

Every plan version was reviewed adversarially by a separate model (Fable 5.1), six rounds so far. Findings are answered one by one in a response document. A finding counts as resolved only when the plan, the code and the pre-registrations agree.

ROUND	REVIEWED → REVISED	FINDINGS	VERDICT	WHAT CHANGED
1	v1 → v2	21	Approve with changes	Hub choice and go/no-go gates, determinism levels split into fresh and replay parity, receipts, pilot designs
2	v2 → v3	19	Approve with changes	D2-replay relabelled as a diagnostic, fallback ladder with an acceptance test, merged checkpoints as certification reference
3	v3 → v4	15	Approve with changes	Architecture converged; pilot hygiene: proxy calibration, surface-form scoring, template clock, three-branch H4 rule
4	v5 → v6	21	Approve with changes	Feasibility model, served artifact A/B, G0 gates, agentic spike, allocation tiers, joint calibration, guards
5	v6 → v6.1	15	Approve with changes	Per-suite cost targets, δ_b from the tested difference, every-variant bound, roadmap order, goal-suite size, spike resolution
6	v6.1 → v6.2	7	Approve with changes	No blocker; structurally converged. Spike gate power, δ_b cap and pooling, switch rule on the spike checkpoint, confirmations that gate
7	v6.2	–	In progress	—

Wave 1 research (determinism)

- Inference numerical determinism and batch invariance
- MoE routing and candidate models
- MoE interpretability and circuits
- Prompt-to-DSL and constrained decoding
- Task-specialised models
- Routing, orchestration and harnesses
- Consistency training and verification

Wave 2 research (goal-compiled MoE)

- Groq LPU and compile-time scheduling
- Token-efficient reasoning, System-1 and System-2
- MoE expert expansion and router training
- Ensembles, System-1 models and classic ML
- Closing the gap: agentic training

Primary evidence

- The user's A3B training report (52 pages), cited as A3B-§ or A3B-ch.N
- Model cards for Qwen3.6-35B-A3B, Qwen3.8-27B and Qwen3.8-Flash-Next
- CPU experiments 00–07 with pre-registrations and JSON results
- 83 unit tests, including a test that every file and symbol the plan cites exists